

MyBicocca

L'app tutto in uno per gli studenti
dell'Università di Milano-Bicocca



Alessandro Autiero | 909697

Alessandro Ferrari | 909536

Federico Giarrusso | 914656

Lorenzo Lupi | 909369

Documentazione del progetto

Indice

Indice	2
Introduzione	3
Strumenti di Lavoro	3
1. Funzionalità	4
1.1 Autenticazione e account	4
1.2 Calendario e orario delle lezioni	4
1.3 Segreteria	5
1.4 Carriera e profilo	5
1.5 Corsi e materiali	5
1.6 Mappa del campus	6
1.7 Biblioteche	6
1.8 Rilevazione delle presenze	6
1.9 Ricerca	6
1.10 Impostazioni e sicurezza	6
2. Architettura	7
2.1 Struttura dei moduli e dei package	8
2.1.1 ui (livello di presentazione)	8
2.1.2 domain (logica)	8
2.1.3 data (accesso ai dati)	8
2.1.4 di (iniezione delle dipendenze)	8
2.1.5 core (funzionalità trasversali)	8
2.1.6 data-api (client di rete)	9
3. Reverse engineering e Sorgenti dati	10
3.1 Misure di sicurezza individuate	10
3.2 Metodologia	10
3.3 Risultati e decisione	11
3.4 Le altre sorgenti dati	11
4. Design	12
4.1 Layout	12
4.2 Palette cromatica e tipografia	15
Conclusioni	16
Punti di Forza	16
Ostacoli Incontrati	16
Sviluppi Futuri	17

Introduzione

MyBicocca è un'applicazione mobile per Android pensata per gli studenti dell'Università di Milano-Bicocca, con l'obiettivo di riunire in un unico strumento i servizi digitali dell'ateneo, oggi distribuiti su piattaforme diverse e non comunicanti tra loro. Lo studente che vuole consultare il libretto, prenotare un appello, scaricare il materiale di un corso, controllare l'orario delle lezioni o trovare un posto in biblioteca deve normalmente passare da Segreterie OnLine, dalla piattaforma Moodle, dall'Agenda Web e da servizi di terze parti, ciascuno con la propria interfaccia e le proprie credenziali.

MyBicocca nasce per superare questa frammentazione e offrire un'unica applicazione coerente, veloce e moderna. L'app supporta le lingue italiana e inglese, adotta il design system Material 3 con tema chiaro e scuro, e mette al centro la rapidità di accesso alle informazioni e la cura dell'esperienza d'uso.

Strumenti di Lavoro

Per lo sviluppo di MyBicocca sono stati utilizzati diversi strumenti, ciascuno con un ruolo preciso nel processo di realizzazione dell'applicazione.

Android Studio è stato l'ambiente di sviluppo, con il linguaggio Kotlin e il toolkit dichiarativo Jetpack Compose. L'interfaccia adotta il design system Material 3, con supporto nativo a tema chiaro e scuro e a più palette selezionabili.

GitHub è la piattaforma di gestione del codice e di collaborazione del gruppo. Il lavoro è organizzato su un ramo stabile, su rami di sviluppo condivisi e su rami dedicati alle singole funzionalità o ai refactoring.

Una scelta architetturale importante è la separazione dei client di rete in una libreria Kotlin autonoma, data-api, costruita come build Gradle inclusa e indipendente dall'app. La libreria incapsula ogni sorgente esterna dietro un'interfaccia tipizzata, così che l'applicazione dipenda dai contratti e non dai dettagli delle singole integrazioni.

La tabella seguente riassume lo stack tecnologico principale.

Ambito	Tecnologia	Ruolo nel progetto
Linguaggio	Kotlin	Linguaggio unico per app e libreria di rete.
Interfaccia	Jetpack Compose, Material 3	UI dichiarativa, tema chiaro e scuro, componenti riutilizzabili.
Navigazione	Navigation 3	Grafo di navigazione tra schermate e sottoschermate.
Dipendenze	Hilt	Iniezione delle dipendenze su tutti gli strati.
Rete	Ktor	Client HTTP della libreria di rete e dell'app.
Persistenza	Room, DataStore, EncryptedSharedPreferences	Cache locale, preferenze e credenziali cifrate.

Osservabilità	Firestore Performance Monitoring	Misura la latenza di ogni chiamata di rete.
Sul dispositivo	ML Kit (voce e barcode), CameraX	Dettatura vocale e scansione del QR per le presenze.
Mappe	MapLibre	Mappa del campus con basemap offline.
Media	Media3, ExoPlayer	Riproduzione delle video lezioni.
Grafici e immagini	Vico, Coil	Grafici della carriera e caricamento immagini.

A questi si aggiungono Jsoup per il parsing dell'HTML dei contenuti Moodle, Haze per gli effetti di sfocatura, Telephoto e Highlights per il visualizzatore di file e di codice, e Shimmer per gli stati di caricamento.

1. Funzionalità

MyBicocca riunisce in un'unica esperienza tutte le attività che lo studente svolge durante la vita universitaria. Le sezioni seguenti descrivono le principali funzionalità, raggruppate per area.

1.1 Autenticazione e account

L'accesso avviene con le credenziali di ateneo. Le credenziali sono salvate sul dispositivo in forma cifrata e non lasciano mai il telefono se non verso i sistemi ufficiali.

Sessioni automatiche: le sessioni verso Esse3 e Moodle vengono costruite e rinnovate in modo trasparente; in caso di scadenza del token la richiesta viene ripetuta automaticamente senza che l'utente debba effettuare di nuovo l'accesso.

Più account: lo studente può registrare e alternare più profili tramite un selettore dedicato, utile a chi gestisce più carriere.

Blocco biometrico: l'app può essere protetta con impronta o riconoscimento del volto, con la possibilità di scegliere il tempo di inattività prima che si ripresenti il lock biometrico.

1.2 Calendario e orario delle lezioni

Il calendario presenta l'orario delle lezioni e gli impegni in tre modalità complementari.

Vista giornaliera: timeline della giornata con indicatore dell'ora corrente e schede degli eventi.

Vista settimanale: griglia della settimana con zoom regolabile e gestione degli eventi sovrapposti.

Vista mensile: agenda del mese con indicatori sui giorni e sezione dei prossimi esami.

In ogni vista sono consultabili i dettagli degli eventi. I dati provengono dall'Agenda Web, servita dal modulo easystaff.

1.3 Segreteria

La sezione segreteria riporta nell'app le funzioni di Segreterie OnLine, organizzate per area amministrativa.

- Prenotazione degli appelli ed esiti degli esami.
- Iscrizioni per anno, piano di studi in consultazione e in modifica.
- Tasse e pagamenti con dettaglio, ISEE e rimborsi.
- Certificati e autocertificazioni, titoli e scadenze.
- Questionari di valutazione della didattica e appuntamenti.

La fonte primaria è l'API REST di Esse3, servita dal modulo `esse3`; per i pochi flussi privi di API REST, come alcune autocertificazioni, interviene il modulo `legacy esse3-scraper`.

1.4 Carriera e profilo

La sezione profilo offre una lettura sintetica e visiva della carriera.

Libretto e statistiche: esami sostenuti, esami per anno, media aritmetica e ponderata, con grafici di avanzamento.

Voto ipotetico: simulazione del voto di laurea a partire dalla media e da ipotesi sui prossimi esami.

Tessera digitale: rappresentazione della tessera dello studente, con la firma resa in stile calligrafico.

1.5 Corsi e materiali

L'integrazione con la piattaforma Moodle dell'ateneo porta nell'app i corsi e tutti i loro contenuti.

Corsi e catalogo: elenco dei corsi seguiti e aggiunta di nuovi corsi dal catalogo.

Materiali e file: navigazione di sezioni e cartelle e visualizzatore integrato per PDF, documenti Office e immagini.

Video lezioni: riproduttore con scelta della qualità e modalità picture in picture, con riproduzione anche in background.

Attività: quiz, compiti, forum, voti e badge del corso.

1.6 Mappa del campus

Una mappa interattiva del campus aiuta a orientarsi tra edifici e aule. La mappa di base è inclusa nell'app, quindi funziona anche senza connessione; sono disponibili l'elenco e il dettaglio degli edifici e dei filtri per categoria. Il rendering si appoggia a MapLibre.

1.7 Biblioteche

Tramite il servizio Affluences, l'app mostra la disponibilità dei posti nelle sale studio e ne consente la prenotazione e l'annullamento. L'integrazione è servita dal modulo `affluences`.

1.8 Rilevazione delle presenze

La presenza in aula può essere registrata inquadrando il codice QR della lezione. Alla rilevazione viene allegata la posizione geografica del dispositivo per la certificazione. La scansione del codice da fuori dall'app può aprire direttamente MyBicocca sulla schermata corretta.

1.9 Ricerca

Una ricerca globale attraversa corsi, esami, edifici e funzioni dell'app. Il motore lavora interamente sul dispositivo, con normalizzazione del testo adatta alla lingua italiana, gestione degli acronimi e tolleranza agli errori di battitura.

1.10 Impostazioni e sicurezza

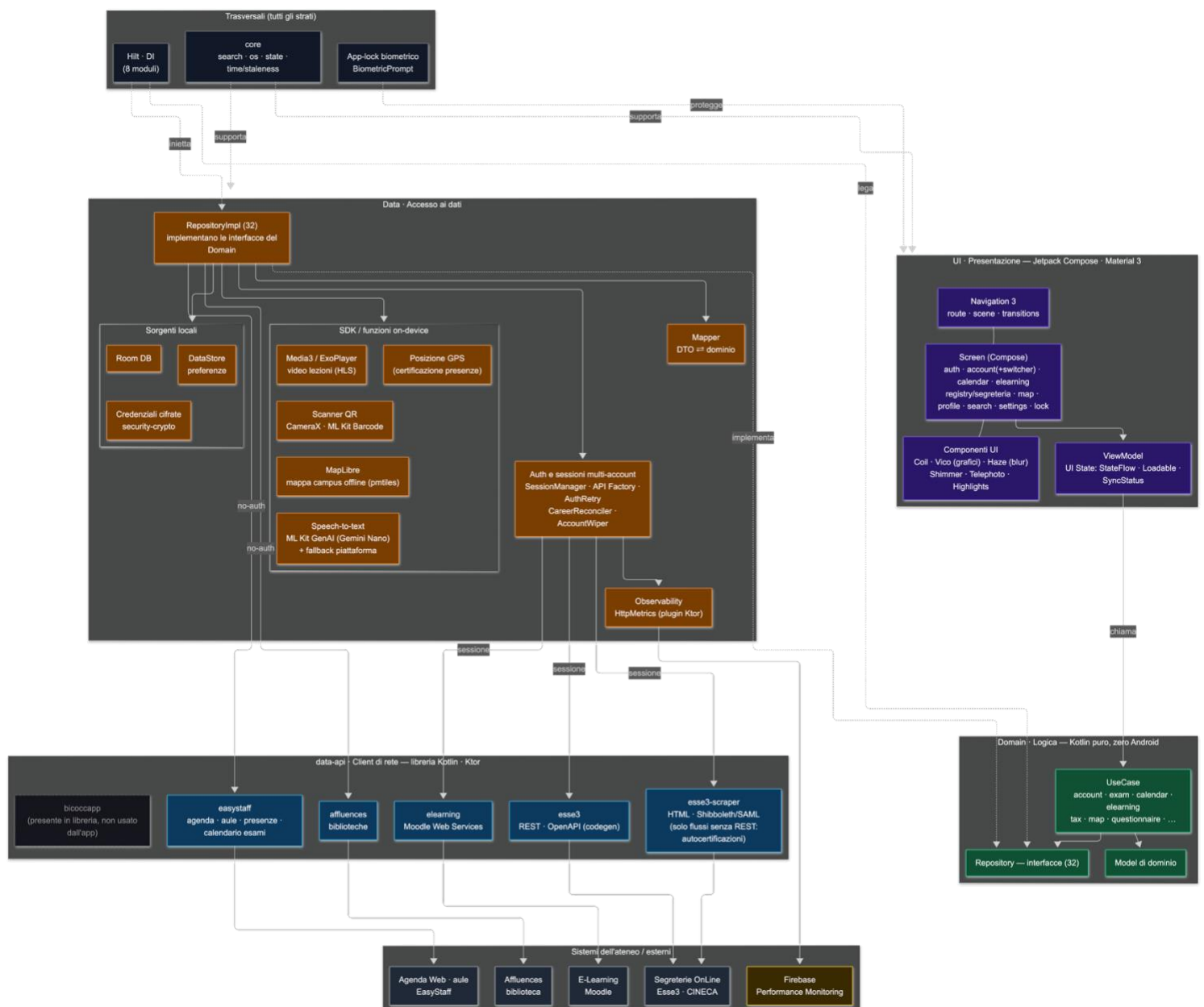
La sezione impostazioni raccoglie la personalizzazione e la sicurezza.

- Scelta del tema tra la palette del brand, il colore dinamico Material You e le varianti con modalità chiara o scura.
- Lingua dell'app tra italiano e inglese.
- Blocco biometrico e associazioni dei file.
- Informazioni sull'app e licenze open source.

2. Architettura

MyBicocca adotta la Clean Architecture con il pattern MVVM e una struttura multimodulo. Il codice è diviso in due build Gradle distinte: l'applicazione Android e la libreria di rete data-api, inclusa come build separata. L'app è a sua volta organizzata in tre strati con dipendenze rivolte verso l'interno, dove lo strato di dominio non conosce né Android né le librerie esterne.

Il flusso è unidirezionale. La schermata osserva lo stato esposto dal ViewModel come StateFlow, il ViewModel invoca i casi d'uso del dominio, i casi d'uso dipendono da interfacce di repository, e le implementazioni nel livello dati realizzano quelle interfacce attingendo alla cache locale e ai client di rete. Le trentadue interfacce di repository del dominio hanno una corrispondente implementazione nel livello dati.



2.1 Struttura dei moduli e dei package

Di seguito sono descritti i moduli e i package principali, seguendo il percorso del dato dalla schermata fino al sistema esterno.

2.1.1 ui (livello di presentazione)

Contiene tutto ciò che riguarda l'interfaccia. Il package `ui/screen` è organizzato per funzionalità (`auth`, `account`, `calendar`, `elearning`, `map`, `profile`, `registry`, `search`, `settings`, `lock`), ciascuna con le proprie sottoschermate, i componenti e lo stato. `ui/navigation` definisce il grafo con Navigation 3, mentre `ui/component` e `ui/theme` raccolgono i componenti riutilizzabili e il tema. I `ViewModel` espongono lo stato come `StateFlow`.

2.1.2 domain (logica)

È il cuore dell'applicazione, scritto in Kotlin puro e indipendente da Android. `domain/model` contiene i modelli di dominio, `domain/repository` le interfacce di accesso ai dati e `domain/usecase` i casi d'uso organizzati per funzionalità, che incapsulano una singola operazione di business.

2.1.3 data (accesso ai dati)

Realizza i contratti del dominio. `data/repository` contiene le implementazioni dei repository, `data/mapper` converte gli oggetti di trasferimento in modelli di dominio, `data/local` gestisce le sorgenti locali (database Room, preferenze DataStore e credenziali cifrate).

Lo strato dati include inoltre sottosistemi dedicati: `data/auth` gestisce sessioni e account (`SessionManager`, factory delle API per sessione, ripetizione automatica dell'autenticazione, riconciliazione della carriera, cancellazione dell'account e blocco biometrico); `data/observability` misura le prestazioni di rete; `data/speech` fornisce la dettatura vocale con ripiego sul riconoscimento di sistema; `data/location` fornisce la posizione per la certificazione delle presenze.

2.1.4 di (iniezione delle dipendenze)

Otto moduli Hilt (`CoreModule`, `DatabaseModule`, `DataStoreModule`, `CredentialsModule`, `RemoteClientModule`, `RepositoryModule`, `SearchModule`, `ApplicationScope`) costruiscono le implementazioni, le legano alle interfacce del dominio e ne governano il ciclo di vita. I client che non richiedono una sessione, come Agenda Web e Affluences, sono forniti qui come singleton; quelli legati all'account, come Esse3 e Moodle, sono costruiti per sessione dalle factory del livello `auth`.

2.1.5 core (funzionalità trasversali)

Raccoglie utilità usate da tutti gli strati: `core/search` è il motore di ricerca in Kotlin (normalizzazione del testo, acronimi, similarità e sottosequenze), `core/os` espone tipo di dispositivo, feedback aptico e picture in picture, `core/state` definisce i tipi di stato comuni e `core/time` le politiche di freschezza della cache.

2.1.6 data-api (client di rete)

La libreria data-api è una build Kotlin separata, basata su Ktor, che racchiude ogni sorgente esterna dietro un'interfaccia tipizzata. Comprende i seguenti moduli.

- `esse3`: client REST per Segreterie OnLine (Esse3, piattaforma CINECA), generato automaticamente dalle specifiche OpenAPI tramite un apposito modulo di code generation.
- `esse3-scraper`: client legacy basato su parsing HTML e sessione Shibboleth con SAML, mantenuto solo per i flussi privi di API REST.
- `elearning`: client per i Web Services della piattaforma Moodle.
- `easystaff`: client per l'Agenda Web (orario, edifici e aule, calendario degli esami e presenze).
- `affluences`: client per la disponibilità e la prenotazione dei posti in biblioteca.
- `bicoccapp`: client dell'app ufficiale ottenuto tramite reverse engineering, presente nella libreria ma non utilizzato dall'applicazione.

Le prestazioni delle chiamate vengono misurate in modo uniforme: un plugin Ktor (`HttpMetrics`) riporta a Firebase Performance Monitoring la latenza di ogni richiesta verso tutti i client, senza modificare le librerie.

3. Reverse engineering e Sorgenti dati

Per capire come l'ateneo espone i propri dati alle applicazioni mobili, il gruppo ha condotto un'analisi dell'app ufficiale BicoccApp. L'obiettivo era individuare le interfacce di rete che l'app utilizza e valutare se riusarle in MyBicocca.

BicoccApp protegge le proprie comunicazioni con diverse contromisure lato client che impediscono l'ispezione del traffico di rete. Per osservare le chiamate verso il server è stato quindi necessario prima comprendere e poi neutralizzare queste protezioni in un ambiente controllato.

3.1 Misure di sicurezza individuate

Un'analisi statica iniziale ha rivelato che BicoccApp è sviluppata con Flutter, il framework multiplatforma di Google, con la logica applicativa nel livello Dart (libreria nativa `libapp.so`) e il motore in `libflutter.so`. L'estrazione delle stringhe dalle librerie native ha permesso di riconoscere le dipendenze di sicurezza impiegate, organizzate su tre livelli di protezione.

- **Rilevamento del root** (`flutter_jailbreak_detection`): l'app rifiuta di avviarsi su dispositivi con permessi di root.
- **Certificate pinning** (`http_certificate_pinning`): l'app accetta solo il certificato atteso, impedendo l'ispezione del traffico tramite un proxy.
- **Autoprotezione a runtime** (Talsec freeRASP): controlli nativi di integrità che segnalano alla parte Dart eventuali manomissioni del dispositivo o dell'app.

3.2 Metodologia

Compresa l'architettura di sicurezza, l'analisi è proseguita combinando strumenti statici e dinamici.

Analisi statica: decompilazione dell'APK con JADX, ispezione del manifest e delle librerie native ed estrazione delle tabelle di stringhe, da cui è emerso l'uso di Flutter e delle tre librerie di protezione.

Analisi dinamica: strumentazione dell'app con Frida. Uno script di ricerca ha intercettato tutti i canali di comunicazione di Flutter (MethodChannel ed EventChannel) tra il livello Dart e quello nativo, registrando metodi, argomenti ed eventi e ricostruendo così il dialogo tra l'app e i suoi moduli di sicurezza.

Proof of concept: individuati i punti in cui transitavano i controlli, è stato sviluppato un prototipo che intercetta i gestori dei canali per neutralizzare le tre protezioni: scarta gli eventi di minaccia di freeRASP, forza a negativo gli esiti del rilevamento root e fa risultare valido il controllo del certificato.

Modulo Magisk e ambiente riproducibile: il prototipo è stato consolidato in un modulo Magisk basato su LSPosed (Xposed con Zygisk), per applicare il bypass in modo stabile senza Frida, e in un emulatore Android automatizzato (Docker con un Pixel 7 Pro API 33, root tramite Magisk, LSPosed e modulo di bypass, accessibile via VNC) per rendere l'intero laboratorio riproducibile.

3.3 Risultati e decisione

Con il pinning neutralizzato è stato possibile ispezionare il traffico HTTPS tramite un proxy e ricostruire l'API del backend. Il risultato è stato documentato come specifica OpenAPI e tradotto in un client Kotlin tipizzato, il modulo `bicoccapp`, con sotto API per autenticazione (OAuth2 e OpenID Connect), profilo, carriera, esami, tasse, calendario, wizard di scelta e campus.

L'analisi ha però portato a una scoperta decisiva: `BicoccApp` è essenzialmente un wrapper attorno all'API di `Esse3`. Conveniva quindi integrare direttamente `Esse3`, più completo e già coperto dall'API REST individuata in precedenza. Per questo il modulo `bicoccapp` è stato mantenuto nella libreria come documentazione della ricerca svolta, ma non viene utilizzato dall'applicazione finale.

3.4 Le altre sorgenti dati

Sorgente	Origine dell'analisi	Modulo
Moodle (corsi)	App Android pubblica di Moodle	<code>elearning</code>
Agenda Web (EasyStaff)	Sito web del servizio	<code>easystaff</code>
Affluences (biblioteche)	App Android di Affluences	<code>affluences</code>
Esse3 (Segreterie)	API REST non documentata da una pagina SwaggerUI non indicizzata	<code>esse3</code>

In tutti i casi il risultato è stato formalizzato e racchiuso dietro un'interfaccia tipizzata, così che l'app dipenda dai contratti delle sorgenti e non dai loro dettagli interni.

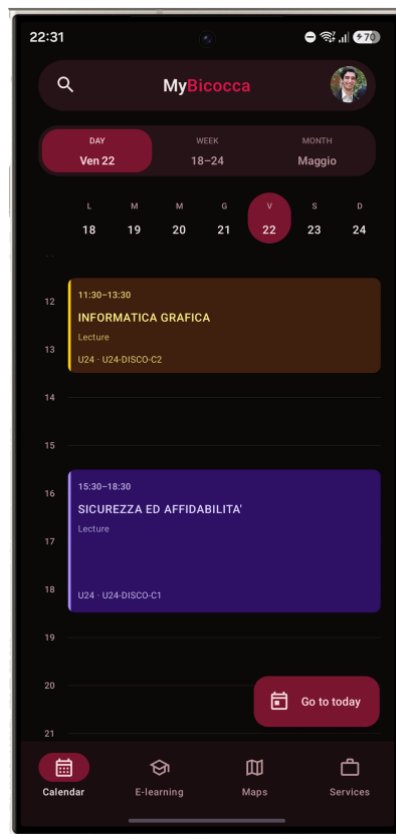
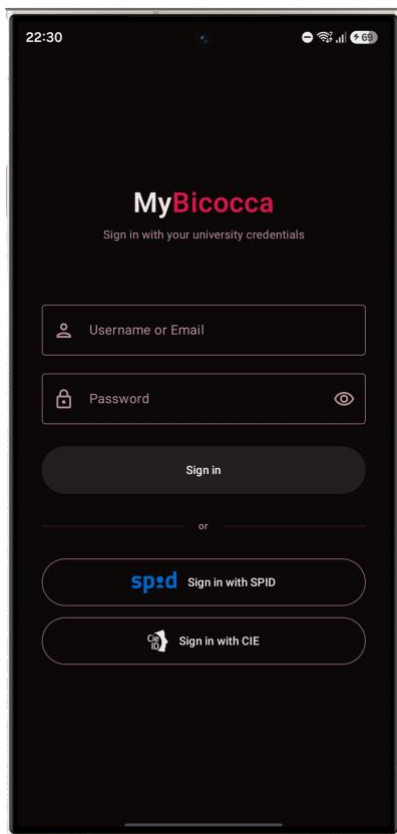
4. Design

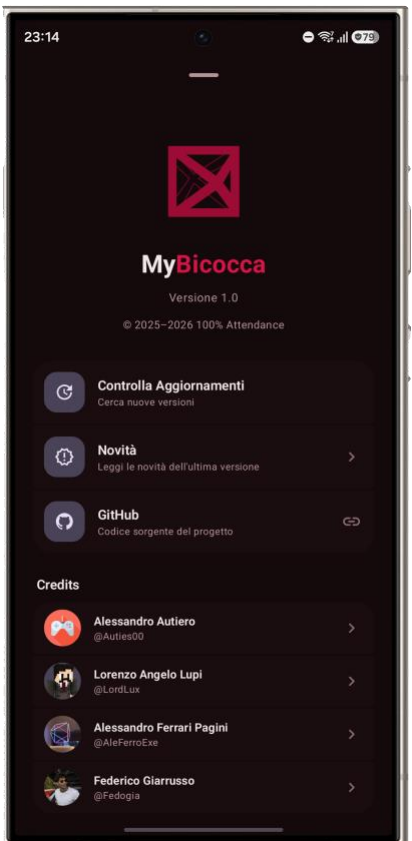
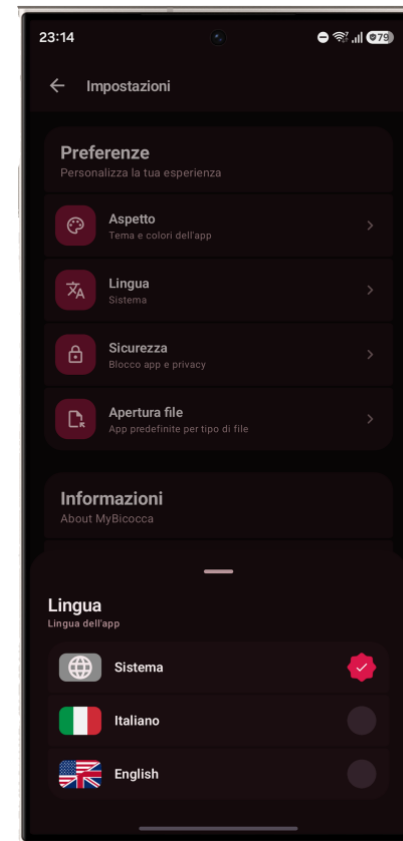
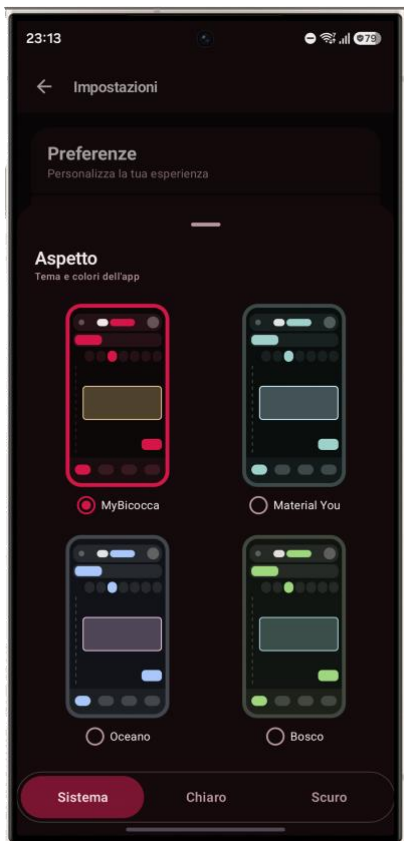
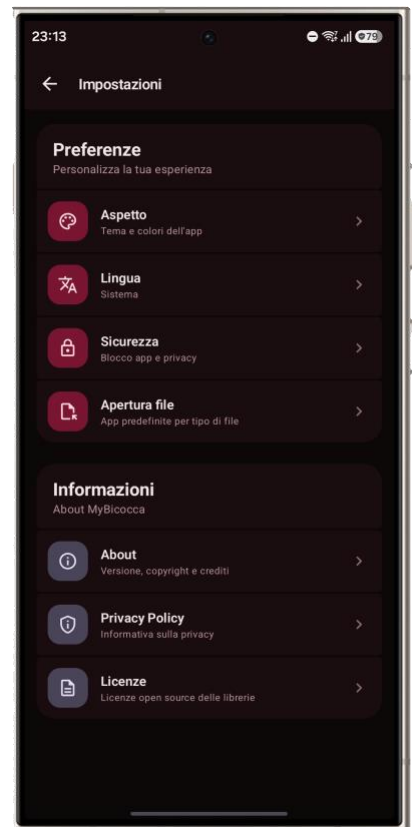
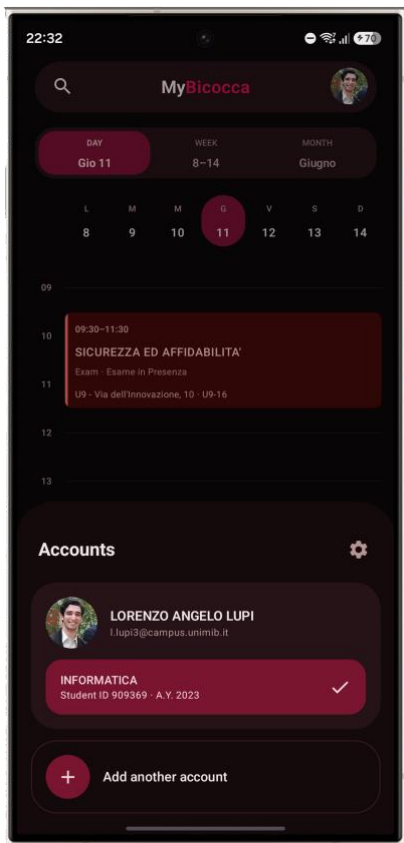
Il design di MyBicocca punta a un'esperienza moderna, leggibile e densa di informazioni, costruita sui principi del Material Design. L'app nasce con un'identità prevalentemente scura, coerente con il marchio dell'ateneo, e privilegia la gerarchia visiva e la rapidità di lettura rispetto a layout rigidi e formali.

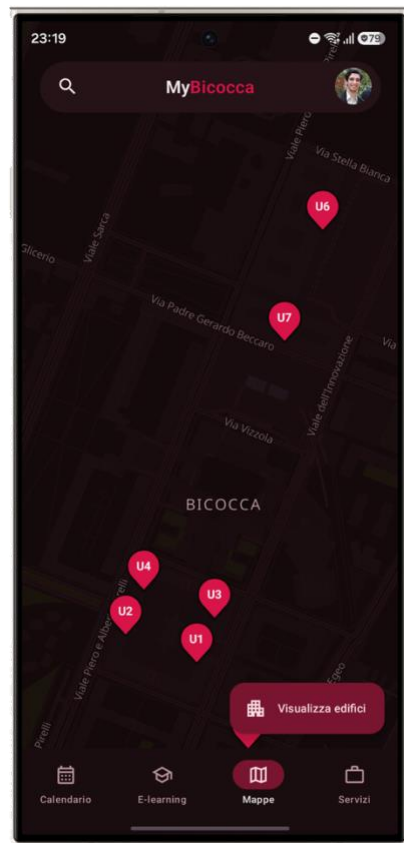
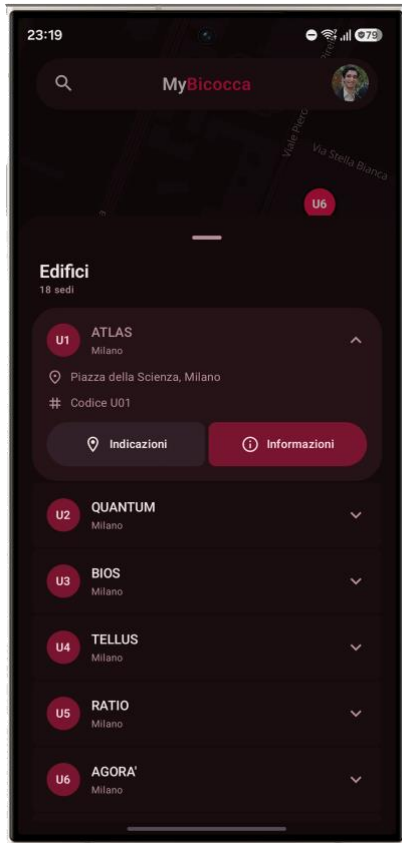
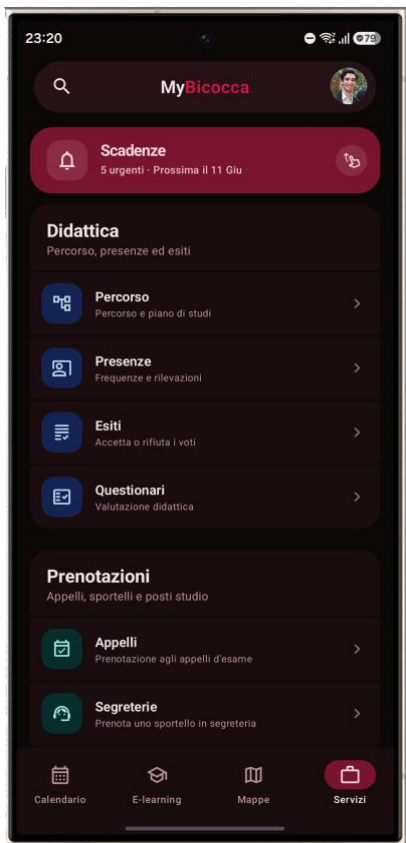
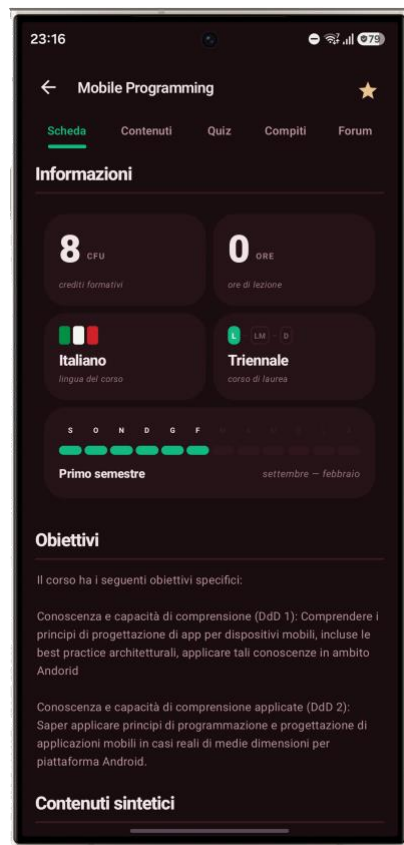
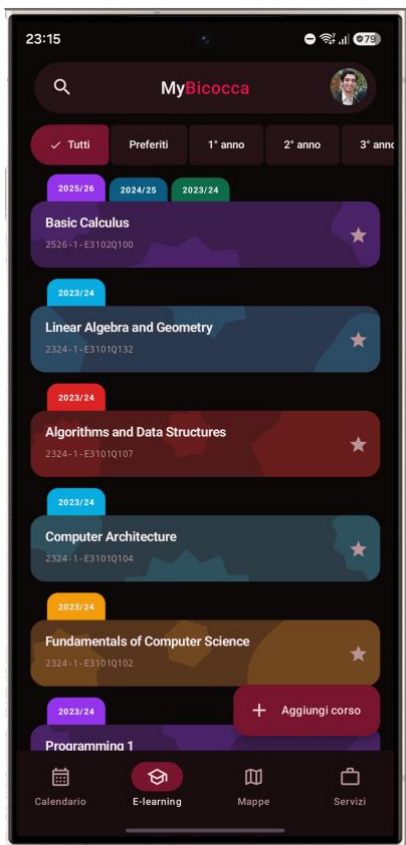
4.1 Layout

La progettazione dei layout segue un approccio centrato sull'utente, che privilegia la chiarezza informativa e la facilità di interazione. Ogni schermata è strutturata per rendere immediata la consultazione dei dati tramite schede ben organizzate e una gerarchia visiva chiara, e si adatta dinamicamente alle diverse dimensioni di schermo.

Le pagine seguenti raccolgono le schermate principali, raggruppate per area funzionale.







4.2 Palette cromatica e tipografia

Il sistema cromatico si basa sugli schemi di colore di Material 3, definiti per intero sia in modalità chiara sia in modalità scura. Il tema predefinito è costruito sul rosso del brand; in alternativa l'utente può scegliere il colore dinamico Material You, che deriva la palette dallo sfondo del dispositivo su Android 12 e versioni successive, e le varianti Oceano, sui toni del blu, e Bosco, sui toni del verde. Alcuni elementi del marchio, come il logo e la tessera dello studente, mantengono colori fissi indipendenti dallo schema attivo, per non alterare l'identità visiva.

Tema predefinito (brand). La tabella riporta i ruoli principali dello schema del brand con il valore esadecimale in modalità chiara e scura. Il rosso primario resta identico nelle due modalità, mentre superfici e testi si invertono per garantire il contrasto.

Ruolo (Material 3)		Chiaro		Scuro
Primario (accento, pulsanti, logo)		#D81648		#D81648
Su primario (testo sul primario)		#FFFFFF		#3D121A
Contenitore primario		#FFD9DF		#7A1530
Secondario		#7A4F56		#E5BCC2
Terziario		#7B5A2E		#E9C28C
Sfondo e superficie		#FFFBFB		#0B0808
Su superficie (testo principale)		#211A1B		#F3E7E9
Contenitore di superficie		#F6EEEF		#1A0D10
Bordo (outline)		#8E7C7F		#7C5A60

Temi alternativi. Per ciascun tema selezionabile sono riportati il colore di accento (primario) e la superficie di sfondo, in modalità chiara e scura. Il tema Material You non ha valori fissi perché deriva i colori dal dispositivo e, dove non disponibile, ricade sul tema del brand.

Tema	Primario chiaro	Primario scuro	Superficie chiara	Superficie scura
Predefinito (brand)	#D81648	#D81648	#FFFBFB	#0B0808
Oceano	#415F91	#AAC7FF	#F9F9FF	#111318
Bosco	#386A20	#9CD67D	#F7FBF1	#111511
Material You	<i>dinamico</i>	<i>dinamico</i>	<i>dinamico</i>	<i>dinamico</i>

Indipendentemente dal tema attivo, il logo MyBicocca resta sempre del rosso del brand #D81648, mentre la tessera digitale dello studente usa un rosso fisso, #F01D59 in chiaro e #9C0C35 in scuro, per preservare la riconoscibilità del marchio.

La tipografia combina un carattere sans serif leggibile per l'interfaccia con tre caratteri dedicati: **Bebas Neue** per i numeri e i dati in evidenza, **JetBrains Mono** per la visualizzazione del codice nel lettore di file, e **Mrs Saint Delafield** per la firma calligrafica sulla tessera digitale.

Conclusioni

MyBicocca dimostra come sia possibile riunire in un'unica applicazione nativa servizi universitari oggi dispersi e tecnologicamente eterogenei. L'adozione della Clean Architecture con MVVM e la separazione dei client di rete in una libreria dedicata hanno prodotto un codice manutenibile e scalabile, mentre l'integrazione di più sorgenti dati ha permesso un'esperienza fluida, ricca e sempre aggiornata.

Punti di Forza

Architettura modulare e a strati: la separazione tra dominio, dati e presentazione e il dominio indipendente da Android rendono il codice testabile e stabile, e facilitano l'aggiunta di nuove funzionalità senza intaccare l'esistente.

Isolamento delle sorgenti esterne: la libreria data-api racchiude ogni integrazione dietro un'interfaccia, così che l'app dipenda dai contratti e non dai dettagli di rete. Il passaggio dallo scraping alle API REST di Esse3 non ha richiesto modifiche all'app.

Client Esse3 generato da OpenAPI: la generazione automatica del codice riduce il lavoro manuale e mantiene il client allineato alle specifiche del servizio.

Esperienza nativa e completa: tre viste calendario, sezione amministrativa, E-learning, riproduttore video, visualizzatore di file, mappa offline, scansione QR e ricerca con dettatura etc, offrono un'esperienza che l'app ufficiale non garantisce.

Funzionamento offline e sicurezza: la cache locale con politiche di freschezza consente la consultazione anche senza rete, mentre credenziali cifrate e blocco biometrico proteggono i dati dello studente.

Osservabilità: Firebase Performance Monitoring misura in modo uniforme la latenza di tutte le chiamate di rete, fornendo dati reali su cui intervenire.

Ostacoli Incontrati

API non documentate: l'API REST di Esse3 è stata individuata in modo fortuito e non è documentata; la mappatura degli acronimi interni della piattaforma ha richiesto un lavoro di interpretazione.

Autenticazione eterogenea: ogni sistema adotta un meccanismo diverso, dalla sessione di Esse3 al token di Moodle fino ai cookie Shibboleth con SAML dello scraper; la loro gestione è centralizzata nel livello auth con factory dedicate.

Gestione di più account: supportare più carriere e più sessioni in parallelo ha richiesto la sincronizzazione per account e la riconciliazione della carriera.

Filtri del calendario: distinguere l'anno di iscrizione e i turni di appartenenza è complesso perché l'informazione è in parte contenuta nei titoli degli eventi.

Migrazione tecnologica: l'adozione di Compose, di Navigation 3 e di un grafo multimodulo ha comportato una curva di apprendimento, partendo anche da un'esperienza pregressa con altri toolkit.

Sviluppi Futuri

L'evoluzione prevista di MyBicocca punta ad ampliare la copertura dei servizi e a rendere l'esperienza ancora più proattiva.

Filtri del calendario: completare il filtraggio degli eventi per anno di iscrizione e per turno, già analizzato in fase di progettazione.

Notifiche: introdurre notifiche per scadenze, esiti degli esami e nuovi materiali pubblicati.

Copertura della segreteria: estendere le funzioni amministrative man mano che il reverse engineering delle sorgenti matura.

Adattamento ai dispositivi: ottimizzare l'interfaccia per tablet e dispositivi pieghevoli, sfruttando la classe di dimensione della finestra già integrata.